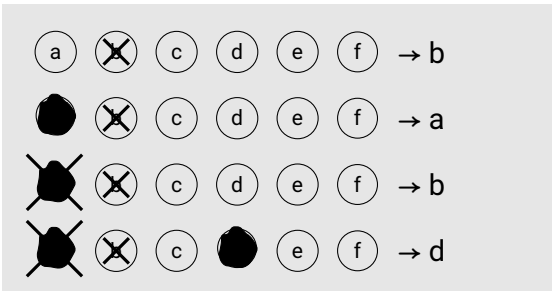
Exercises

1	2	3	4	5	6

1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9
0	0	0	0	0	0	0

Surname, First name

Algorithmic Design (BCS1540)
Exam



Answer multiple-choice questions as shown in the example.

Program: BSc Computer Science
Course code: BCS1540
Examiners: Steven Chaplick & David Mestel
Date/time: 20/05/2026 12:00-14:00hr
Format: Closed book
Allowed aids: Pens (black or dark blue), NO calculator allowed
Instructions to students:

- The exam consists of 5 questions on 16 pages, with a total of 60 points.
- Fill in your name and student ID number on the cover page and tick the corresponding numerals of your student number in the table (top right cover page).
- Answer every question in the reserved space below the question. **Do <u>not</u> write outside the reserved space or on the back of pages, this will not be scanned and will NOT be graded!** As a last resort if you run out of space, use the extra answer space at the end of the exam.
- *In no circumstance write on or near the QR code at the bottom of the page!*
- Ensure that you properly motivate your answers.
- Only use black or dark blue pens, and write in a readable way. Do not use pencils.
- Answers that cannot be read easily cannot be graded and may therefore lower your grade.
- If you think a question is ambiguous, or even erroneous, and you cannot ask during the exam to clarify this, explain this in detail in the space reserved for the answer to the question.
- If you have not registered for the exam, your answers will not be graded, and thus handled as invalid.
- You are not allowed to have a communication device within your reach, nor to wear or use a watch.
- You have to return all pages of the exam. You are not allowed to take any sheets, even blank, home.
- Good luck!

Greedy: Interview Scheduling

Input: A set of n applicants $A(1), \ldots, A(n)$ where, for each $i \in \{1, \ldots, n\}$, the availability of $A(i)$ is given as a time interval $[A(i).s, A(i).f]$ starting at time $A(i).s$ and finishing at time $A(i).f)$ with $A(i).s < A(i).f$.

There is also **one** interviewer wanting to interview as many applicants as possible. The interviewer has setup m possible start times $t_1, t_2, \ldots, t_m$ for the interviews. An interview lasts 1 unit of time. These times are given in increasing order, and are spaced at least one unit apart, ie,

. for each $j \in \{1, \ldots, m-1\}$, $t_j + 1 \leq t_{j+1}$.

Applicant i can be **scheduled** for an interview at time t_j if:

. $t_j \geq A(i).s$ and $t_j + 1 \leq A(i).f$, and no other applicant is scheduled at time t_j.

Objective: Schedule a maximum size subset of the applicants.

Note: The applicants are also ordered by finish time: $A(1).f \leq A(2).f \leq \ldots \leq A(n).f)$.

Example Input: $[A[1].s, A[1].f]$=$[0, 3]$, $[A[2].s, A[2].f]$=$[0, 4]$, $[A[3].s, A[3].f]$=$[3, 6]$; and t_1=0, t_2=4, t_3=6. An optimal solution is to schedule $A[2]$ at time t_1 and $A[3]$ at time t_2. Note that only $A[3]$ can be scheduled at time t_2 and none of the applicants can be scheduled at time t_3.

Two greedy rules: GREEDYA (given next), and GREEDYB (on the next page) are given to be considered.

- GREEDYA: A k-partial solution is a schedule for the **last** k interview time slots, ie, for each $j \in \{m-k+1, \ldots, m\}$, we have decided whether or not there is an applicant scheduled at time t_j.
  Rule: Start with $k = 0$. Consider the next starting time (t_{m-k}). If no unscheduled applicant can be scheduled at time t_{m-k}, skip it. Otherwise, among applicants that can be scheduled at time t_{m-k}, schedule one with the earliest finish time at time t_{m-k}.

3p **1a** Provide a counterexample to the optimality of GREEDYA.

- GREEDYB: A k-partial solution is a schedule for the **first** k interview time slots, ie, for each $j \in \{1, \ldots, k\}$, we have decided whether or not there is an applicant scheduled at time t_j.
  Rule: Start with $k = 0$. Consider the next starting time (t_{k+1}). If no unscheduled applicant can be scheduled at time t_{k+1}, skip it. Otherwise, among applicants that can be scheduled at time t_{k+1}, schedule one with the earliest finish time at time t_{k+1}.

8p **1b** Prove that GREEDYB is optimal.

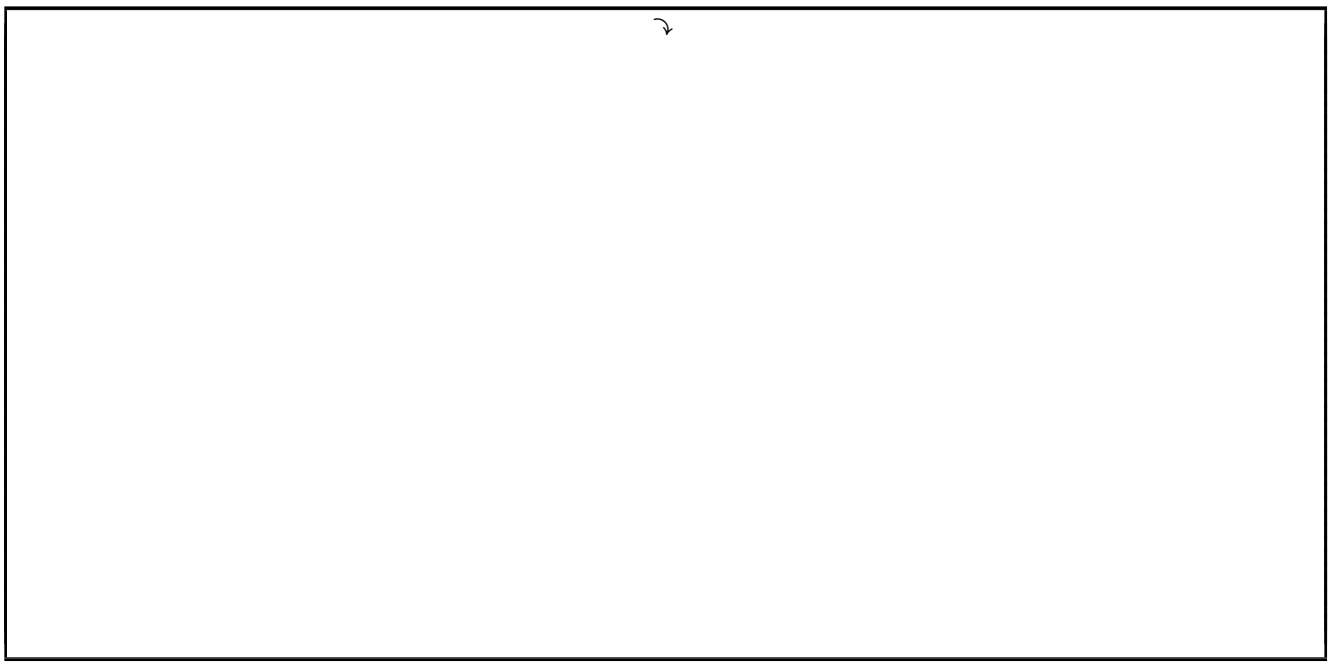
5p **1c** Describe how you would implement GREEDYB and analyze the runtime.

The Master Theorem

3p **2a** I have an algorithm that solves a problem instance of size n by solving 12 instances of size $n/3$, plus additional work taking time $n \log n$. Give the recurrence satisfied by my algorithm, and solve it using the Master Theorem to find the asymptotic runtime of my algorithm.

3p **2b** Suppose now I improve my algorithm to use only 9 instances of size $n/3$, plus the same additional work as before. Give the recurrence and solve it to find the runtime of my new algorithm.

↳



Constructing broadcast towers of varying cost to cover a highway.
Input: The highway is continuous and has length $\ell > 0$: starting at position 0 and ending ℓ km away. There are n candidate towers that can be built.
Each candidate tower $i \in \{1, \ldots, n\}$ has an interval $[a_i, b_i]$ $(a_i < b_i)$ **and a positive cost** c_i **in EUROs** such that if the tower is built, **we pay the cost** c_i and the highway is covered from and including position a_i up to and including position b_i.
Objective: Select a **minimum cost** set S of towers so that every position on the highway is covered by some tower. The set S is allowed to contain towers with overlapping intervals.
You may assume that every position on the highway can be covered by some candidate tower, i.e., for every value $x \in [0, \ell]$ there is some tower i such that $x \in [a_i, b_i]$.

Note: The input has the form $([a_1, b_1], c_1), \ldots ([a_n, b_n], c_n)$ where $b_1 \leq b_2 \leq \ldots \leq b_n$.
Example Input: $\ell = 10$ and 6 candidate towers: $([0, 3], 2), ([0, 5], 4), ([3, 8], 1), ([2, 9], 2), ([4, 10], 2), ([7, 10], 1)$.
An optimal solution would be to use $([0, 3], 2)$, $([3, 8], 1)$, and $([7, 10], 1)$. The cost is $4 = 2+1+1$.

Your Task: Provide a dynamic programming algorithm to optimally solve this problem as outlined next. For full credit, your algorithm must run in time polynomial in terms of the input size.

3p **3a** Define the table `OPT(...)` of subproblems: (i) what parameter(s) `(...)` should it have, (ii) what value is stored in each entry, and (iii) what entry in your table represents an optimal solution to the problem?

8p **3b** State and justify the correctness of a recurrence that you can use to fill in the table. Don't forget the base cases!

↴

5p **3c** Analyze the runtime needed to implement the algorithm.
Make sure to state the size of your table and the time to compute each entry.
Your analysis must also include outputting an optimal **set** of towers and not only the optimal value.

NP-completeness

Recall the two problems Independent-Set and Vertex-Cover, whose optimisation versions are as follows:
Independent-Set: given a graph G, what is the largest set of vertices X such that no two vertices are joined by an edge?
Vertex-Cover: given a graph G, what is the smallest set of vertices Y such that for every edge of G, one of its two endpoints is in Y?

4p **4a** Formulate the decision problem versions of these two problems.

2p **4b** Prove that Independent-Set$\in$NP.

↳

4p **4c** Suppose we know that Vertex-Cover is NP-hard, and we want to prove that Independent-Set is also NP-hard. Give an appropriate polynomial-time reduction from one of these two problems to the other that allows us to conclude this. Make sure you describe your reduction clearly and precisely.

Backtracking, LP & branch-and-bound

PostNL have asked you to help design an algorithm to plan routes for their delivery drivers.

They have D drivers, each of whom is allowed to drive up to L kilometers, starting and ending at the depot. There are N locations to deliver to, each of which must be visited by a driver. They have a table of distances $c_{i,j}$ of distances between locations (for $0 \leq i, j \leq n$ the distance between location i and location j is $c_{i,j} = c_{j,i}$; the depot is location 0).

5p **5a** Describe (with pseudocode or an equivalent level of detail and precision) a backtracking algorithm to solve this problem.

4p **5b** Formulate the problem as an integer linear program, using variables $x_{d,k,i,j}$, where

$$x_{d,k,i,j} = \begin{cases} 1 & \text{if the } k\text{th trip segment made by driver } d \text{ is from location } i \text{ to location } j \\ 0 & \text{otherwise} \end{cases}.$$

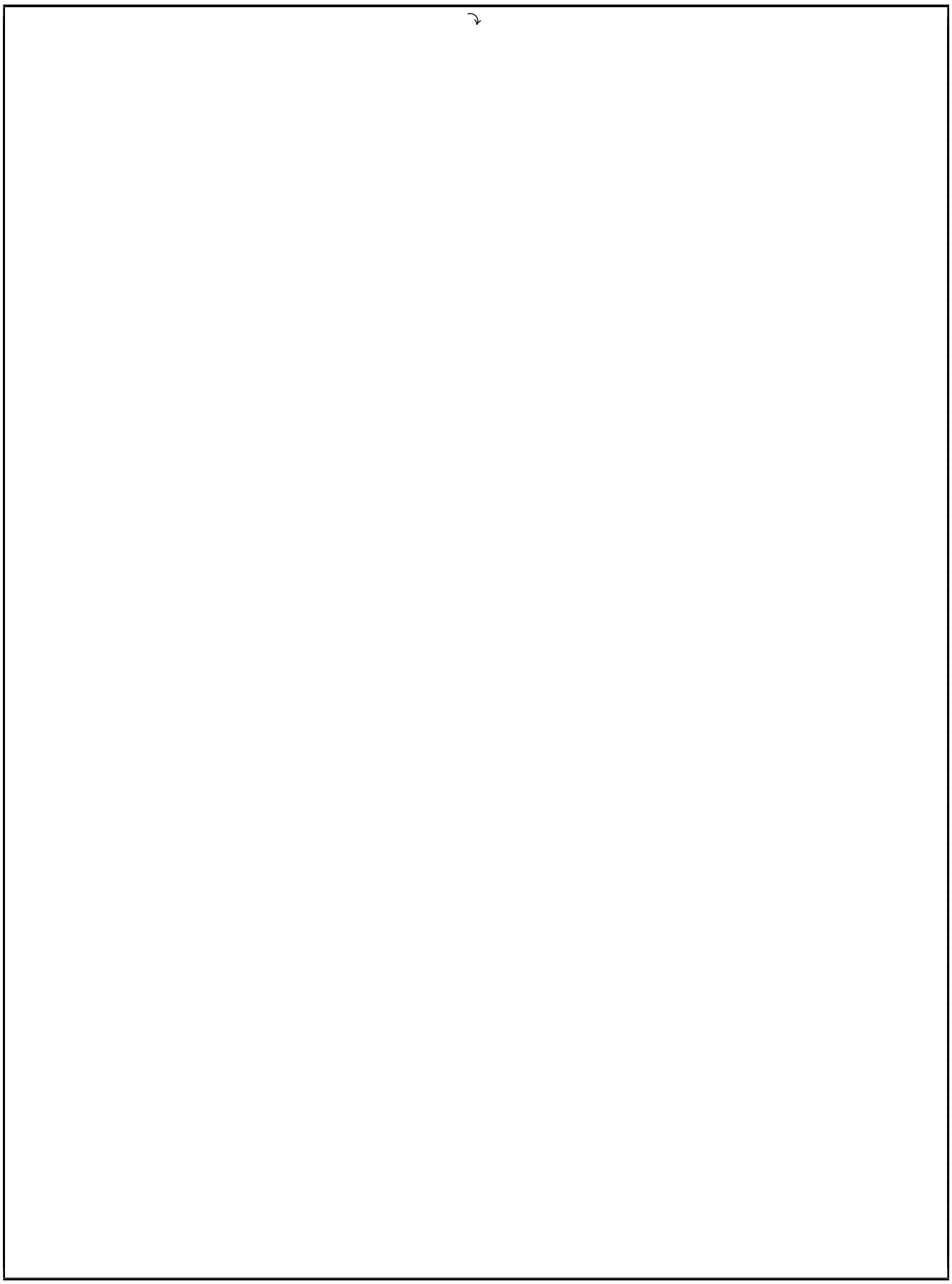
For example, if driver 3 delivers to locations 7, 3 and 11 in that order, then we would have $x_{3,1,0,7} = x_{3,2,7,3} = x_{3,3,3,11} = x_{3,4,11,0} = 1$, and all other $x_{3,d,k,i,j} = 0$.

3p **5c** State what is meant by the *fractional relaxation* of this ILP, and explain precisely how to update your algorithm from part a to use branch-and-bound with this fractional relaxation.

Extra Space

Any answers that did not fit in the space provided above can be given here. For each answer, you must clearly state the question you are answering.

6



This page is left blank intentionally